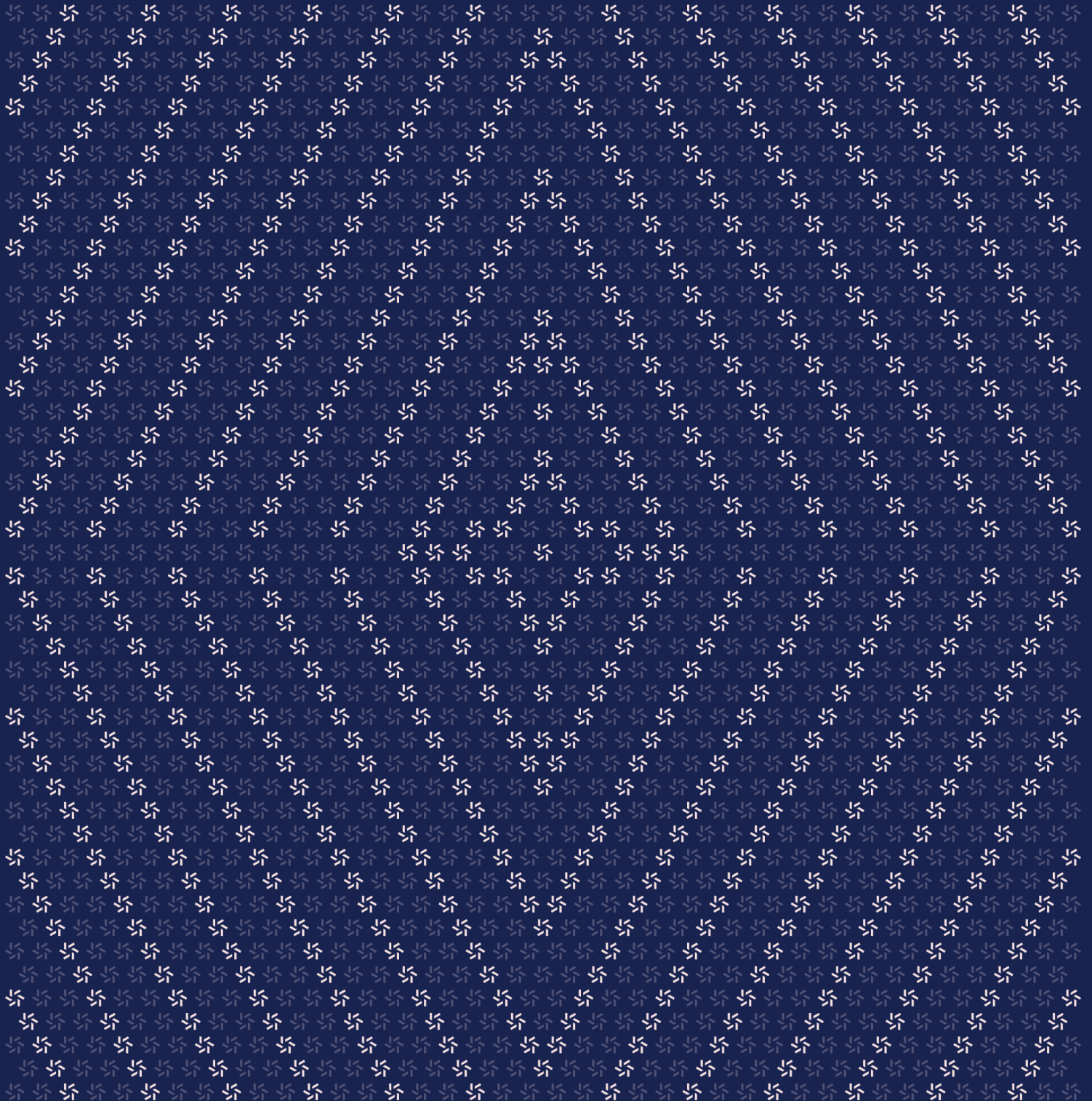


August 11, 2026

Tenbin

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	6
2. Introduction	7
2.1. About Tenbin	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	10
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. Using <code>convertToAssets</code> will overstate adapter value for fee-charging vaults	11
3.2. Cross-chain UNSTAKE bypasses restricted-account enforcement	13
3.3. Floor rounding in the rate-bound check breaks the signed-quote invariant	15
3.4. Unguarded sender casting is inconsistent with the contract's defensive pattern	17
3.5. Missing identity validation allows self-referential adapter deployment	19
3.6. Stale cooldown ownership entries are not cleared after terminal actions	21
3.7. Obsolete fee role constants remain after fee functionality was removed	23

4.	Patch Review	24
4.1.	Changes in CrossChainERC4626Adapter.sol	24

5.	Threat Model	26
5.1.	BebopHook.sol	26
5.2.	CrossChainERC4626Adapter.sol	32
5.3.	GenericMorphoAdapter.sol	47
5.4.	GenericMorphoAdapterFactory.sol	52

6.	Assessment Results	54
6.1.	Disclaimer	54

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Tenbin Labs from August 4th to August 11th, 2026. During this engagement, Zellic reviewed Tenbin's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does GenericMorphoAdapter accurately report the realizable value of its ERC-4626 position and enforce appropriate slippage bounds during allocation and deallocation?
 - Can an unauthorized party move, rescue, strand, or redirect the underlying assets or vault shares managed by GenericMorphoAdapter or cause a parent vault to use an unintended adapter?
 - Does BebopHook ensure that each Bebop RFQ fill and Tenbin Controller order agree on the action, token pair, amounts, payer, recipient, authorized market maker, and signed exchange rate?
 - Do Bebop-backed mint and redemption transactions settle atomically without leaving reusable approvals, residual balances, or user funds stuck in the hook or router?
 - Does CrossChainERC4626Adapter correctly decode and validate the source chain, sender, action, tokens, beneficiary, and cooldown identifier for every cross-chain message?
 - Do authorization changes, including account restrictions, remain effective throughout the cooldown life cycle?
 - Can deposit, cooldown, unstake, cancellation, delivery, or failed-message recovery paths reach an unrecoverable state, strand assets, or process the same position more than once?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

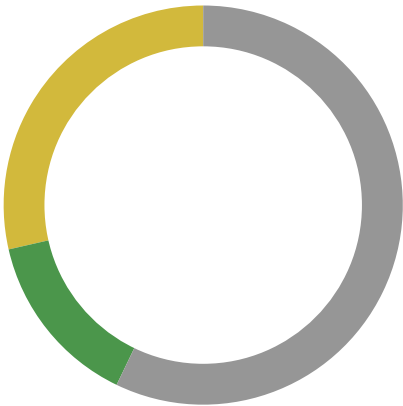
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Tenbin contracts, we discovered seven findings. No critical issues were found. Two findings were of medium impact, one was of low impact, and the remaining findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	2
Low	1
Informational	4



2. Introduction

2.1. About Tenbin

Tenbin Labs contributed the following description of Tenbin:

Tenbin is an asset token issuance platform with the goal of creating liquid, composable financial assets. Assets in the Tenbin protocol are backed by two positions: off-chain futures contracts and on-chain collateral. The off-chain hedging system maintains a delta one exposure of an underlying asset. The on-chain collateral is used to earn low-risk yield. So long as the on-chain yield equals or exceeds the off-chain funding costs, the protocol is able to peg Tenbin assets to the spot price of the real asset.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Tenbin Contracts

Type	Solidity
Platform	EVM-compatible
Target	tenbin-contracts
Repository	https://github.com/tenbinlabs/tenbin-contracts ↗
Version	633540ee51d7a069f4a5a53feccd8e3498d1ff14
Programs	bebop/BebopHook.sol bebop/IBebopHook.sol morpho/GenericMorphoAdapter.sol morpho/GenericMorphoAdapterFactory.sol
Target	Only changes between e3f8016...cde2faf
Repository	https://github.com/tenbinlabs/cross-chain-erc4626-vault ↗
Version	cde2faffec4470c9a3f33103efd6a52d027cd696
Programs	src/CrossChainERC4626Adapter.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.1 person-weeks. The assessment was conducted by two consultants over the course of six calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Kritsada Dechawattana
✈ Security Engineer
kritsada@zellic.io ↗

Chongyu Lv
✈ Security Engineer
chongyu@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

August 4, 2026 Start of primary review period

August 5, 2026 Kick-off call

August 11, 2026 End of primary review period

3. Detailed Findings

3.1. Using convertToAssets will overstate adapter value for fee-charging vaults

Target	GenericMorphoAdapter		
Category	Integration Risks	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `realAssets()` function values the adapter's ERC-4626 share balance by calling `vault.convertToAssets(balance)`. ERC-4626 requires `convertToAssets()` to exclude fees, while `previewRedeem()` must include withdrawal fees. Consequently, when the configured ERC-4626 vault charges a withdrawal fee, `realAssets()` reports more assets than the adapter could obtain by redeeming its shares.

```
function realAssets() public view returns (uint256) {
    uint256 balance = vault.balanceOf(address(this));
    return vault.convertToAssets(balance);
}
```

Impact

The parent Vault V2 includes each adapter's `realAssets()` value when calculating its total assets and share price. The overstated value can therefore be reflected in the parent vault's accounting before the withdrawal fee is realized. A user redeeming against available liquidity may receive assets based on this gross value, while a later deallocation realizes the fee and leaves the shortfall to the remaining vault share holders. The discrepancy is proportional to the underlying vault's withdrawal fee and occurs only when `GenericMorphoAdapter` is configured with a fee-charging ERC-4626 vault.

Recommendations

Replace `vault.convertToAssets(balance)` with `vault.previewRedeem(balance)` so that the reported value includes withdrawal fees and reflects the assets obtainable by redeeming the adapter's current share balance.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit [267e13be](#).

3.2. Cross-chain UNSTAKE bypasses restricted-account enforcement

Target	CrossChainERC4626Adapter		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

When processing a cross-chain COOLDOWN, `CrossChainERC4626Adapter._processPayload()` calls `cooldownShares()` directly:

```

} else if (action == Action.COOLDOWN) {
    if (!redeemsEnabled) revert RedeemsDisabled();
    // The sent token must be vault minted token
    if (inputToken != vault) revert InvalidActionInputToken();
    (outputAmount, cooldownId)
    = IStakedAsset(vault).cooldownShares(inputAmount);

```

The adapter is therefore the immediate caller, so `StakedAsset` records the cooldown under the adapter address — `processMessage()` separately records the source-chain sender only in `cooldownAccounts`:

```

if (action == Action.COOLDOWN) {
    cooldownAccounts[message.sourceChainSelector][id]
    = bytes32(message.sender);
    emit CrossChainCooldownCreated(message.sourceChainSelector,
    bytes32(message.sender), outputAmount, id);

```

For a later UNSTAKE, `processMessage()` verifies only that the message comes from the sender that originally created the cooldown. It does not check whether that sender has since become restricted:

```

if (action == Action.CANCEL_COOLDOWN || action == Action.UNSTAKE) {
    if (cooldownAccounts[message.sourceChainSelector][cooldownId] !=
    bytes32(message.sender)) {
        revert InvalidCooldown();
    }
    if (message.destTokenAmounts.length != 0)
    revert InvalidTokenCount(message.destTokenAmounts.length);
}

```

The adapter then completes the cooldown as both the caller and receiver:

```
} else if (action == Action.UNSTAKE) {  
    if (!redeemsEnabled) revert RedeemsDisabled();  
    // Extract amount to be unstaked with specific cooldown Id  
    IStakedAsset.Cooldown memory cooldown  
    = IStakedAsset(vault).cooldowns(address(this), id);  
  
    outputAmount = cooldown.assets;  
  
    IStakedAsset(vault).unstake(address(this), id);  
    outputToken = assetToken;
```

As a result, the StakedAsset restriction checks apply to the adapter rather than to the source user. The released assets are subsequently delivered to the beneficiary without checking the source user's current restriction status.

Impact

Consider a user who starts a cooldown while unrestricted and is added to the restricted list before the cooldown expires. After the waiting period, the same user can submit a data-only cross-chain UNSTAKE message. The unrestricted adapter completes its own StakedAsset cooldown and transfers the released underlying assets to the selected beneficiary.

The restricted user can therefore recover all assets associated with a cooldown created before the restriction was applied, bypassing the intended restricted-account process. The issue does not allow the user to claim another account's cooldown.

Recommendations

Associate each adapter-owned cooldown with a canonical source-chain identity and check that identity's current restriction status before processing UNSTAKE or CANCEL_COOLDOWN. The beneficiary should also be validated under the applicable destination-chain restriction policy. Because supported chains may use different identity formats, the restriction check should use an explicit chain-aware identity mapping rather than relying on the adapter's local address.

Remediation

This issue has been acknowledged by Tenbin Labs, and fixes were implemented in the following commits:

- [9d002a3d](#) ↗
- [35a25d85](#) ↗

3.3. Floor rounding in the rate-bound check breaks the signed-quote invariant

Target	BebopHook		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `verifyHookData()` function enforces that the fill rate does not exceed the market maker's signed quote by computing `Math.mulDiv(outputAmount, data.quoteInputAmount, inputAmount)` and comparing the result with `data.quoteOutputAmount`. The three-argument `Math.mulDiv()` rounds down. When `outputAmount * quoteInputAmount` is not evenly divisible by `inputAmount`, the computed quotient can equal `quoteOutputAmount` even though the exact rate is greater than the signed bound.

```
if (Math.mulDiv(outputAmount, data.quoteInputAmount, inputAmount) >
    data.quoteOutputAmount) {
    revert InvalidAmount();
}
```

Specifically, when

$$\text{outputAmount} * \text{quoteInputAmount} = \text{quoteOutputAmount} * \text{inputAmount} + r$$

for some $0 < r < \text{inputAmount}$, floor division returns exactly `quoteOutputAmount`. The comparison is therefore false, and the fill is accepted despite violating the exact inequality.

Impact

A fill can settle at a rate that is strictly above the market maker's signed limit. The normalized value discarded by floor division is less than one smallest unit of `quoteOutputAmount`; the corresponding excess in `outputAmount` is less than `inputAmount / quoteInputAmount` units. Although this is a bounded rounding discrepancy rather than an unbounded loss path, it breaks the signed-rate invariant enforced by the hook.

Recommendations

Round the quotient up before comparing it with the signed limit. For positive amounts, the rounded-up result exceeds `quoteOutputAmount` whenever the exact rational value exceeds it,

while retaining `Math.mulDiv()`'s full-precision multiplication:

```
if (Math.mulDiv(outputAmount, data.quoteInputAmount, inputAmount) > data.  
    quoteOutputAmount) {  
  if (  
    Math.mulDiv(outputAmount, data.quoteInputAmount, inputAmount, Math.  
      Rounding.Ceil)  
    > data.quoteOutputAmount  
  ) {  
    revert InvalidAmount();  
  }  
}
```

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit [28998612](#).

3.4. Unguarded sender casting is inconsistent with the contract's defensive pattern

Target	CrossChainERC4626Adapter		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The refund-related paths normalize the CCIP message sender through `_normalizeToBytes32()`, which requires the sender encoding to be exactly 32 bytes and otherwise reverts with `InvalidSenderAddressFormat`.

However, `processMessage()` converts `message.sender` directly with `bytes32(message.sender)` when authorizing terminal cooldown actions, recording cooldown ownership, and emitting `CrossChainCooldownCreated`:

```

if (action == Action.CANCEL_COOLDOWN || action == Action.UNSTAKE) {
    if (cooldownAccounts[message.sourceChainSelector][cooldownId] != bytes32(
        message.sender)) {
        revert InvalidCooldown();
    }
}

if (action == Action.COOLDOWN) {
    cooldownAccounts[message.sourceChainSelector][id] = bytes32(message.
        sender);
    emit CrossChainCooldownCreated(message.sourceChainSelector, bytes32(
        message.sender), outputAmount, id);
}

```

A direct conversion accepts non-32-byte values by right-padding shorter inputs and truncating longer inputs, rather than rejecting them consistently with the refund paths.

Impact

Shorter sender values produce noncanonical `bytes32` identities, although the same sender is converted consistently during cooldown creation and completion and therefore does not become unable to complete its cooldown. Longer sender values are more concerning because distinct

encodings with the same first 32 bytes resolve to the same cooldown identity and could authorize one identity to act on another identity's cooldown.

The currently supported EVM and SVM sender formats are exactly 32 bytes, so this behavior has no direct impact under the supported configuration. It is a defensive-consistency and forward-compatibility issue if a future or misconfigured lane supplies another sender length.

Recommendations

Normalize `message.sender` once with `_normalizeToBytes32()` before cooldown authorization, storage, or event emission, and reuse the normalized value throughout `processMessage()`.

Because `processMessage()` is invoked as an external self-call inside the `ccipReceive()` try-catch, `InvalidSenderAddressFormat` would be handled as a failed message rather than reverting the entire CCIP delivery.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit [39f8c98c](#).

3.5. Missing identity validation allows self-referential adapter deployment

Target	GenericMorphoAdapter		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The GenericMorphoAdapter constructor verifies that vault_ and parentVault_ use the same underlying asset but does not require them to be distinct addresses. If both parameters contain the parent Vault V2 address, the AssetMismatch check passes and the adapter deploys successfully.

```

constructor(address parentVault_, address vault_, address owner_)
    Ownable(owner_) {
        parentVault = IVaultV2(parentVault_);
        vault = IERC4626(vault_);
        asset = parentVault.asset();
        // Passes trivially if vault_ == parentVault_: same address, same asset()
        if (vault.asset() != parentVault.asset()) revert AssetMismatch();
        [...]
    }

```

The adapter must then be registered through the timelocked addAdapter() path before any impact occurs. Once registered, the following call sequence recurses until the transaction runs out of gas:

```

VaultV2.accrueInterestView()
-> GenericMorphoAdapter.realAssets()
-> parentVault.convertToAssets()
-> VaultV2.previewRedeem()
-> VaultV2.accrueInterestView()

```

Vault V2 uses the transient storage variable firstTotalAssets as an intratransaction short circuit. However, accrueInterest() sets it only after accrueInterestView() returns. The recursive call therefore occurs before the short circuit is initialized.

The deposit(), mint(), withdraw(), redeem(), and allocate() paths call accrueInterest() directly and consequently revert. Deallocation also fails because GenericMorphoAdapter.deallocate() calls withdraw() on the same parent vault, which enters the recursive accounting path.

Impact

Core deposit, mint, withdrawal, redemption, and allocation operations become unavailable while the self-referential adapter remains registered. Ordinary ERC-20 share transfers and approvals are unaffected. The curator can recover the affected operations by scheduling and executing `removeAdapter()`, subject to its configured timelock. No direct loss of funds was demonstrated.

Recommendations

Add an explicit identity check in the constructor to reject self-referential deployments at the point of creation:

```
constructor(address parentVault_, address vault_, address owner_)
    Ownable(owner_) {
    if (vault_ == parentVault_) revert SelfReferentialVault();
    parentVault = IVaultV2(parentVault_);
    vault = IERC4626(vault_);
    asset = parentVault.asset();
    if (vault.asset() != parentVault.asset()) revert AssetMismatch();
    [...]
}
```

A dedicated error such as `SelfReferentialVault` would improve diagnostic clarity. The same guard should be mirrored in `GenericMorphoAdapterFactory.createGenericMorphoAdapter()` so that the deployment path rejects the invalid configuration before instantiating the adapter.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit [b5a6bc92](#).

3.6. Stale cooldown ownership entries are not cleared after terminal actions

Target	CrossChainERC4626Adapter		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `cooldownAccounts` mapping records the originating sender of a cross-chain COOLDOWN action so that only that sender can later submit UNSTAKE or CANCEL_COOLDOWN for the same `(sourceChainSelector, cooldownId)` pair. Neither terminal action removes the entry after successful execution, so the ownership record persists indefinitely.

When a COOLDOWN is initiated, ownership is recorded:

```
[...]
if (action == Action.COOLDOWN) {
    cooldownAccounts[message.sourceChainSelector][id]
    = bytes32(message.sender);
    emit CrossChainCooldownCreated(...);
}
[...]
```

After UNSTAKE or CANCEL_COOLDOWN completes successfully, `cooldownAccounts[sourceChainSelector][cooldownId]` is never deleted. The authorization entry remains set to the original sender even though the underlying cooldown has been consumed.

Impact

There is no direct asset impact because `StakedAsset` deletes the underlying cooldown during either terminal action. A replay reaches `StakedAsset.unstake()` or `StakedAsset.cancelCooldown()` and reverts with `ZeroCooldownAssets()` before any additional assets can be released. The remaining entry is stale authorization state with no operational use and can make the public ownership mapping appear to describe an active cooldown after that cooldown has ended.

Recommendations

Delete the `cooldownAccounts` entry after `_processPayload()` successfully completes an `UNSTAKE` or `CANCEL_COOLDOWN`. If a later delivery step reverts, transaction atomicity restores both the vault cooldown and the ownership entry.

```
function processMessage(Client.Any2EVMMessage calldata message)
    external
    onlySelf
    onlyValidChain(message.sourceChainSelector)
{
    [...]
    (address outputToken, uint256 outputAmount, uint256 id) =
        _processPayload(inputToken, inputAmount, payload.minimumOut, action,
            cooldownId);

    if (action == Action.CANCEL_COOLDOWN || action == Action.UNSTAKE) {
        delete cooldownAccounts[message.sourceChainSelector][cooldownId];
    }

    emit PayloadProcessed(message.messageId, inputToken, outputToken,
        inputAmount, outputAmount);
}
```

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit [4f45d807](#).

3.7. Obsolete fee role constants remain after fee functionality was removed

Target	CrossChainERC4626Adapter		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The scoped changes remove the adapter's fee-configuration functions, fee-collection logic, fee-withdrawal function, and constructor role grants. However, CrossChainERC4626Adapter still declares the corresponding FEE_SETTER_ROLE and FEE_COLLECTOR_ROLE constants:

```
bytes32 public constant FEE_SETTER_ROLE = keccak256("FEE_SETTER_ROLE");
bytes32 public constant FEE_COLLECTOR_ROLE = keccak256("FEE_COLLECTOR_ROLE");
```

No function in the scoped contract references either role. Because the constants are public, they also remain exposed as ABI getters even though there is no fee operation for either role to authorize. The contract additionally retains NatSpec references to the removed assetFees mechanism.

Impact

The obsolete constants have no direct security impact. They add unnecessary ABI surface and bytecode and may lead operators or integrators to believe that the adapter still supports fee configuration and collection.

Recommendations

Remove FEE_SETTER_ROLE, FEE_COLLECTOR_ROLE, and the remaining assetFees documentation references if fee management is no longer supported. If fee management is intended to remain, restore the complete fee configuration, collection, withdrawal, and role-assignment paths rather than retaining role identifiers alone.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit [4ce82c1b](#).

4. Patch Review

The purpose of this section is to document the exact commit diff of the codebase that was considered in scope for this audit.

As requested by Tenbin Labs, we focused on the changes made between commit [e3f80160](#) and commit [cde2fafe](#).

4.1. Changes in CrossChainERC4626Adapter.sol

This section summarizes the in-scope changes to `CrossChainERC4626Adapter.sol`, including cross-chain cooldown support, the immutable single-vault model, removal of the fee system, payload-processing changes, and related event updates.

Cross-chain cooldown support

The adapter was extended to support staked-asset cooldown life-cycle operations via Chainlink CCIP, replacing the previous deposit/redeem binary with a four-action dispatch model.

- A new `Action` enum with variants `DEPOSIT`, `COOLDOWN`, `UNSTAKE`, and `CANCEL_COOLDOWN` was added.
- The `Payload` struct's `target` field was replaced with `actionAndId`.
- A new `cooldownAccounts` mapping (`uint64 chain => uint256 id => bytes32 account`) binds each cooldown ID to the originating chain and sender, enforcing that only that account can later `UNSTAKE` or `CANCEL_COOLDOWN`.
- The `processMessage` function now validates token counts by `Action`; `CANCEL_COOLDOWN` and `UNSTAKE` require zero inbound tokens, while `DEPOSIT` and `COOLDOWN` require exactly one.
- After `_processPayload` returns, `COOLDOWN` stores the initiator in `cooldownAccounts` and emits `CrossChainCooldownCreated`. All other actions follow the existing return-to-source or local-delivery path.

Vault model refactoring (single-vault, immutable)

The dynamic target allowlist was replaced with a single immutable vault configured at deployment.

- Two new immutable variables, `VAULT` and `ASSET`, are set in the constructor.
- The `enabledTargets` allowlist mapping was removed along with `setTargetEnabled` and the `TargetEnabled` event.
- The constructor's `feeSetter` and `feeCollector` parameters were replaced with a single `vault` parameter. Role grants for `FEE_SETTER_ROLE` and `FEE_COLLECTOR_ROLE` were also removed.
- A new `InvalidVault` error guards against a zero-address vault at construction.

Fee system removal

The entire protocol fee infrastructure was removed.

- The `assetFees` and `collectedFees` mappings and all related accounting logic were deleted.
- Along with the `AssetFeeSet` and `FeeWithdrawn` events, `setAssetFee`, `setAssetFees`, and `withdrawFee` were removed.
- The errors `InvalidFeeSetter`, `InvalidFeeCollector`, `InsufficientFeeBalance`, `FeeConfigLengthMismatch`, and `FeeExceedsAmount` were removed.
- `preview` also dropped the `vaultTarget` parameter (now using immutable `VAULT`) and all fee deduction logic, returning raw vault preview output.

Payload processor refactoring (`_processTarget` → `_processPayload`)

The core vault interaction routine was redesigned for the new action model.

- `_processTarget` was renamed `_processPayload`. Its signature changed from `(target, inputToken, inputAmount, minimumOut, applyAssetFee, assetFeeDestinationChainSelector)` to `(inputToken, inputAmount, minimumOut, action, id)`.
- The function now uses immutable `VAULT` and `ASSET` instead of deriving the asset via `IERC4626(target).asset()` on each call.
- `DEPOSIT` validates `inputToken == ASSET`, approves the vault, and then calls `IERC4626(vault).deposit`.
- `UNSTAKE` reads the pending amount from `IStakedAsset(vault).cooldowns(address(this), id)` and then calls `unstake`. Output is taken from the stored cooldown, not a return value.
- `COOLDOWN` validates `inputToken == VAULT` and then calls `IStakedAsset(vault).cooldownShares(inputAmount)`, which returns both the asset amount and the new cooldown ID.
- `CANCEL_COOLDOWN` calls `IStakedAsset(vault).cancelCooldown(id)` and returns the share token as output.
- The function now returns a third value, `cooldownId`, used by `processMessage` to populate `cooldownAccounts`.
- All fee-skimming logic was removed. Three new errors — `InvalidAction`, `InvalidCooldown`, and `InvalidActionInputToken` — cover the new validation paths.

Event changes

- `TargetProcessed` was renamed `PayloadProcessed`. The target indexed parameter was removed since the vault is now implicit.
- A new `CrossChainCooldownCreated` event was added, emitting `chainSelector`, `initiator` (as `bytes32`), `assets`, and the `cooldown id`.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. BebopHook.sol

Function: `bebopHook(address makerAddress, bytes data, Swap[] swaps)`

This router-only entry point executes one full-fill Bebop self-swap as a Tenbin Controller mint or redeem. It decodes the market maker's hook and issuer data; checks the maker, allowance, order, quoted rate, tokens, and single swap leg; pulls the input from the router; executes the Controller order; verifies the produced output; and returns the quoted output to the router.

Inputs

- `makerAddress`
 - **Control:** Full control.
 - **Constraints:** The caller must be the immutable Bebop router, and the supplied address must equal the hook's current `marketMaker`.
 - **Impact:** Identifies the market maker whose signed quote and delegated Controller authority are accepted for the execution.
- `data`
 - **Control:** Full control.
 - **Constraints:** The bytes must ABI-decode first as `HookData` and then as `IssuerData`. The decoded action must be mint or redeem, and every decoded order, token, amount, rate, payer, recipient, and signature field must satisfy the hook and Controller checks.
 - **Impact:** Defines the Controller action, input and output tokens, signed quote amounts, order, signatures, context, and approval used to move assets.
- `swaps`
 - **Control:** Full control.
 - **Constraints:** The array must contain exactly one leg. That leg must be a self-swap denominated in the decoded output token, with equal maker and taker amounts and a nonzero maker amount that matches the Controller order's output amount.
 - **Impact:** Supplies the Bebop fill amounts and output-token identity used to enforce a fill-or-kill Controller execution and determine how much output is returned to the router.

Branches and code coverage

Intended branches

- ☑ A full-fill mint should pull collateral from the router, mint the exact asset amount through the Controller, and return the output to the router without leaving inventory on the hook.
- ☑ A full-fill redeem should pull the asset token from the router, redeem the exact collateral amount through the Controller, and return the output without leaving inventory on the hook.
- ☑ A single matching self-swap should be accepted for both mint and redeem orders.
- ☑ Normal mint and redeem execution should measure the Controller-produced balance and transfer the exact expected output to the router.

Negative behavior

- ☑ Revert when the caller is not the configured router.
- ☑ Revert when makerAddress is not the configured market maker.
- ☑ Revert when the swap array contains zero legs or more than one leg.
- ☑ Revert when the router allowance does not equal the fixed Controller order input amount.
- ☑ A scaled partial mint or redeem fill through the Bebop router should revert atomically without consuming user funds or router and Controller nonces.
- ☑ Revert when the action, order type, payer, recipient, token pair, swap token, or swap amount is inconsistent.
- ☑ Revert when the Controller produces less than the quoted output amount.
- ☑ Revert when the actual mint or redeem output does not equal the Controller order output.
- ☐ Exercise a Controller call that produces more than the quoted output, and verify how the resulting surplus held by the hook is handled.
- ☐ Revert when malformed data fails either ABI decode with a test that asserts atomic preservation of router and hook balances.
- ☐ Exercise a token or Controller callback that attempts to reenter bebopHook and verify that no independent allowance or balance can be reused.

Function: rescueToken(address token, address to)

This owner-only recovery function transfers the hook's entire balance of an ERC-20 token to a chosen recipient. It is intended to recover tokens accidentally left after an otherwise atomic hook execution.

Inputs

- token
- **Control:** Full control.

- **Constraints:** The caller must be the owner, and the token address must be nonzero and implement a transfer behavior accepted by SafeERC20.
 - **Impact:** Selects the ERC-20 balance that is removed in full from the hook.
- to
 - **Control:** Full control.
 - **Constraints:** The recipient must be nonzero.
 - **Impact:** Receives the hook's entire balance of the selected token.

Branches and code coverage

Intended branches

- ☒ The owner can transfer the full balance of an accidentally held ERC-20 token to a nonzero recipient.

Negative behavior

- ☒ Revert when the caller is not the owner.
- ☒ Revert when the recipient is the zero address.
- ☐ Revert when the token address is zero.
- ☐ Exercise a token whose transfer returns false or reverts, and verify that no partial recovery state is committed.

Function: `setDelegateStatus(address maker, bool status)`

This owner-only function directly changes whether a signer may authorize Controller orders whose payer is the hook. Unlike `setMarketMaker`, it does not update the hook's `marketMaker` state.

Inputs

- `maker`
 - **Control:** Full control.
 - **Constraints:** The caller must be the hook owner. Enabling delegation additionally requires `maker` to be an approved Controller signer; disabling delegation accepts any address.
 - **Impact:** Selects the Controller signer whose delegated authority for the hook is granted or revoked.
- `status`
 - **Control:** Full control.
 - **Constraints:** `true` enables the delegation, and `false` disables it.
 - **Impact:** Determines whether the selected signer may authorize orders paid

by the hook at the Controller layer.

Branches and code coverage

Intended branches

- ☒ The owner can enable Controller delegation for an approved signer.
- ☐ The owner can disable an existing delegation.

Negative behavior

- ☒ Revert when the caller is not the owner.
- ☐ Revert when enabling delegation for an address that is not an approved Controller signer.
- ☐ Exercise disabling delegation for the current `marketMaker`, and verify the resulting hook-to-Controller state mismatch is prevented or explicitly accepted.
- ☐ Exercise enabling a signer other than `marketMaker`, and verify whether direct Controller use outside the hook is intended.

Function: `setMarketMaker(address maker)`

This owner-only function replaces the market maker accepted by `bebopHook`. It revokes the old maker's Controller delegation, updates the hook state, grants delegation to the new maker, and emits `MarketMakerUpdated` in one transaction.

Inputs

- `maker`
 - **Control:** Full control.
 - **Constraints:** The caller must be the hook owner, the address must be nonzero, and the Controller requires the address to be an approved signer before delegation can be enabled.
 - **Impact:** Becomes the only maker identity accepted by the hook and a Controller delegate authorized to sign orders paid by the hook.

Branches and code coverage

Intended branches

- ☒ The owner can update `marketMaker` to a nonzero approved signer and emit `MarketMakerUpdated`.
- ☐ The old maker's Controller delegation should be revoked, and the new maker's delegation should be enabled in the same successful transaction.

- ☐ A revert from either Controller delegation update should roll back the hook state and both delegation changes atomically.

Negative behavior

- ☒ Revert when the caller is not the owner.
- ☒ Revert when the new maker is the zero address.
- ☐ Revert without changing state when the new maker is not an approved Controller signer.

Function: `verifyHookData(HookData data, IController.Order order, Swap swap, uint256 inputAmount, uint256 outputAmount)`

This internal validation function binds the Bebop hook data and single swap leg to the Controller order. It validates the action, nonzero amounts, signed quote rate, hook custody addresses, mint-or-redeem token direction, and self-swap shape before any input tokens are pulled from the router.

Inputs

- `data`
 - **Control:** Full control.
 - **Constraints:** The action must encode Controller mint or redeem. The quote amounts must be nonzero, the signed rate bound must hold, and the declared input and output tokens must match the selected Controller order direction.
 - **Impact:** Supplies the maker-authorized action, token pair, and quote rate that bound the execution.
- `order`
 - **Control:** Full control.
 - **Constraints:** Its type must match `data.action`; its payer and recipient must both be the hook; and its collateral and order tokens must match the hook data in the direction required by mint or redeem.
 - **Impact:** Defines the Controller operation and the amounts and assets that the Controller will mint or redeem.
- `swap`
 - **Control:** Full control.
 - **Constraints:** Both `makerToken` and `takerToken` must be equal to each other and to `data.outputToken`, and `makerAmount` must equal `takerAmount`.
 - **Impact:** Binds the Bebop self-swap leg to the output token and amount expected from the Controller.
- `inputAmount`

- **Control:** Full control.
- **Constraints:** The value is derived from the input token allowance granted by the router and must be nonzero before validation succeeds.
- **Impact:** Forms the denominator of the executed output-to-input rate and represents the amount later pulled from the router.
- `outputAmount`
 - **Control:** Full control.
 - **Constraints:** The value is derived from `swap.makerAmount`, must be nonzero, and must satisfy the signed quote rate bound.
 - **Impact:** Forms the numerator of the executed rate and determines the amount the hook expects the Controller to produce and sends back to the router.

Branches and code coverage

Intended branches

- ☒ Matching mint hook data, Controller order fields, and self-swap data should pass validation.
- ☒ Matching redeem hook data should reverse the input and output token direction and pass validation.
- ☒ An executed rate equal to the signed quote should pass validation.

Negative behavior

- ☒ Revert when the action is unsupported or does not match the Controller order type.
- ☒ Revert when `inputAmount` is zero.
- ☒ Revert when the executed output-to-input rate exceeds the signed quote by a whole-unit amount.
- ☒ Revert when the order payer or recipient is not the hook.
- ☒ Revert when the mint or redeem token direction does not match the hook data.
- ☒ Revert when the swap is not a one-token self-swap or its maker and taker amounts differ.
- ☐ Exercise a fractional rate just above the signed bound where floor rounding in `Math.mulDiv` produces the same integer result.
- ☐ Exercise zero `outputAmount`, zero `quoteInputAmount`, and zero `quoteOutputAmount` independently so each remaining amount guard is reached directly.

5.2. CrossChainERC4626Adapter.sol

Function: `_processPayload(address inputToken, uint256 inputAmount, uint256 minimumOut, Action action, uint128 id)`

This private vault dispatcher executes one of four actions against the immutable vault: deposit underlying for shares, start a share cooldown, unstake an expired cooldown into underlying, or cancel a cooldown back into shares. It returns the output token and amount, returns a new ID for cooldown creation, and enforces nonzero output and the caller-specified minimum.

Inputs

- `inputToken`
 - **Control:** Full control.
 - **Constraints:** The value is derived from the single CCIP-delivered token for deposit and cooldown actions. It must equal ASSET for a deposit and VAULT for cooldown creation; unstake and cancellation receive the zero-address default because they are tokenless.
 - **Impact:** Selects and authorizes the token consumed by token-bearing vault actions.
- `inputAmount`
 - **Control:** Full control.
 - **Constraints:** The value is derived from the CCIP-delivered amount and is required to be nonzero by `processMessage` for deposit and cooldown actions.
 - **Impact:** Determines how many underlying tokens are deposited or how many vault shares enter cooldown.
- `minimumOut`
 - **Control:** Full control.
 - **Constraints:** The actual output amount returned or read from the vault must be greater than or equal to this value.
 - **Impact:** Protects the source user from accepting fewer output shares or underlying assets than specified.
- `action`
 - **Control:** Full control.
 - **Constraints:** `processMessage` derives the enum from the payload and rejects values greater than `CANCEL_COOLDOWN` before calling this function.
 - **Impact:** Selects deposit, cooldown creation, unstake, or cooldown cancellation and determines which processing-enable flag applies.
- `id`
 - **Control:** Full control.

- **Constraints:** The low 128 bits of the source payload supply the value. It is used only for unstake and cancellation, which first require the source chain and sender to match `cooldownAccounts` for that ID.
- **Impact:** Selects the adapter-owned vault cooldown whose underlying assets are released or whose shares are reminted.

Branches and code coverage

Intended branches

- ☒ `DEPOSIT` should deposit the full delivered underlying amount and return the minted vault token and share amount.
- ☒ `COOLDOWN` should pass the delivered shares to `cooldownShares`, return the underlying-denominated amount and new cooldown ID, and intentionally return no immediately deliverable token.
- ☒ `UNSTAKE` should read the adapter-owned cooldown assets, complete the expired cooldown to the adapter, and return the underlying token and amount.
- ☒ `CANCEL_COOLDOWN` should cancel the adapter-owned cooldown and return the vault share token and reminted share amount.
- ☒ A successful action whose output equals `minimumOut` should pass.

Negative behavior

- ☒ Revert when a deposit does not supply `ASSET` or a cooldown does not supply `VAULT`.
- ☒ Revert when deposits are disabled before executing a deposit.
- ☒ Revert when unstake is attempted before the cooldown expiry through the vault dependency.
- ☐ Revert when cooldown creation or unstake is attempted while redemptions are disabled and when cancellation is attempted while deposits are disabled.
- ☐ Revert when the vault action returns zero output.
- ☐ Reach and assert `MinimumOutputNotMet` when the output is below `minimumOut`, and verify that the external vault call and all token movements roll back. In `CrossChainERC4626Adapter.t.sol`, `test_failure_minimum_output_not_met_marks_message_failed()` selects `CANCEL_COOLDOWN` without an authorized cooldown and therefore reverts earlier with `InvalidCooldown`.
- ☐ Exercise a nonexistent, already completed, or already canceled cooldown ID for both terminal actions.
- ☐ Enforce the current restricted-account policy for the source user before completing `UNSTAKE`.

Function: `ccipReceive(Client.Any2EVMMessage message)`

This router-only entry point receives an inbound CCIP message and isolates vault processing in an external self-call. Successful processing emits `MessageSucceeded`; any processing revert is caught, the message is marked failed, refund-relevant metadata and an outbound chain-family snapshot are stored, and `MessageFailed` is emitted so the delivered tokens remain recoverable.

Inputs

- `message`
 - **Control:** Partial control. The remote sender controls the application payload and the tokens and amounts submitted to CCIP, while the CCIP router authenticates and supplies the message ID, source-chain selector, sender encoding, and delivered-token records.
 - **Constraints:** The immediate caller must be the immutable CCIP router. The router supplies the message ID, authenticated source selector and sender bytes, payload, and delivered token amounts; `processMessage` applies the application-level chain, payload, action, token, amount, authorization, and vault checks.
 - **Impact:** Selects the cross-chain action and assets to process. On failure, its ID keys the recovery state, its source and sender determine the refund route, its delivered token list determines refundable assets, and its payload may provide a local recovery address.

Branches and code coverage

Intended branches

- ☒ A valid inbound message should complete its vault action without creating a failed-message record.
- ☒ A processing revert should be caught, mark the message BASIC, and store the refund-relevant source, sender, token amounts, and local recovery address.
- ☒ Oversized or malformed payload data should not be copied into failure storage, and malformed-length payloads should leave the local recovery address zero.
- ☒ The refund chain family should be snapshotted from configuration or inferred from a 32-byte EVM or SVM sender when the source selector is unconfigured.

Negative behavior

- ☒ Revert when the immediate caller is not the configured CCIP router.
- ☐ Assert the exact `MessageSucceeded` and `MessageFailed` events for the corresponding terminal path.
- ☐ Redelivering a previously seen `messageId` should be tested against the router's delivery guarantees so successful execution or an existing failed record cannot be duplicated or overwritten unexpectedly.

- Exercise a failure after an otherwise valid vault action reaches the outbound return leg, and verify that the entire self-call rolls back while the inbound tokens remain recoverable. In `CrossChainERC4626Adapter.t.sol`, `test_failure_bridge_back_after_deposit_holds_inbound_asset()` selects `CANCEL_COOLDOWN` without an authorized cooldown and therefore fails before the vault action.

Function: `processMessage(Client.Any2EVMMessage message)`

This self-call-only function validates and executes the application logic of an inbound CCIP message. It decodes the action and optional cooldown ID, enforces token-count and cooldown-owner rules, dispatches the configured vault action, records new cooldown ownership, and either delivers the resulting token locally or bridges it back to the source chain.

Inputs

- `message`
 - **Control:** Partial control. The remote sender controls the application payload and the tokens and amounts submitted to CCIP, but the function accepts the message only through the adapter's self-call after router delivery, and the router authenticates the envelope fields.
 - **Constraints:** The immediate caller must be the adapter itself, and the source selector must be enabled; data must be exactly 128 bytes and decode as `Payload`. Bits 128 through 135 of `actionAndId` must encode an action from zero through three, while the low 128 bits encode the cooldown ID. Deposit and cooldown actions require exactly one nonzero delivered token amount; unstake and cancellation require no delivered tokens and a stored cooldown account equal to `bytes32(message.sender)`. Local and EVM return beneficiaries must be canonical right-aligned EVM addresses.
 - **Impact:** Determines the vault action, cooldown identity, source-account authorization, minimum output, beneficiary, local-versus-cross-chain delivery, input assets, and final recipient of the resulting tokens.

Branches and code coverage

Intended branches

- ☑ A deposit should accept the configured underlying token and deliver the minted shares locally.
- ☑ A deposit should bridge minted shares back to configured EVM and SVM source chains with the correct wire encoding.
- ☑ A cooldown should consume vault shares, create a vault cooldown owned by the adapter, and map the returned ID to the authenticated source chain and sender.

- ☒ An authorized cooldown owner should be able to cancel and receive shares locally.
- ☒ After expiry, an authorized cooldown owner should be able to unstake and receive underlying assets locally or bridge them to the source chain.
- ☒ Independent cooldowns should map distinct returned IDs to their respective source senders.

Negative behavior

- ☒ Revert when the function is called by an address other than the adapter itself.
- ☒ Revert when the source chain is disabled or the payload length is not exactly 128 bytes.
- ☒ Revert when a token-bearing action supplies more than one delivered token.
- ☒ Revert when an unstake originates from a different configured source chain or sender than the stored cooldown owner.
- ☒ Revert when a local EVM beneficiary is not canonically encoded in 32 bytes.
- ☐ Revert when the decoded action is greater than three.
- ☐ Revert when a deposit or cooldown carries zero tokens or when unstake or cancellation carries any token.
- ☒ Revert when a deposit is delivered with a zero token amount before vault processing.
- ☐ Reject non-32-byte message . sender encodings before cooldown ownership is stored or compared.
- ☐ Prevent a source account that becomes restricted after creating a cooldown from completing UNSTAKE while the restriction remains active.
- ☐ Clear cooldownAccounts after successful unstake or cancellation, or explicitly test and document why stale terminal ownership records are retained.
- ☐ Reject or explicitly ignore nonzero actionAndId bits above bit 135.

Function: recoverFailedMessageLocally(bytes32 messageId)

This nonreentrant recovery function lets the local refund address encoded in a failed message claim every nonzero delivered-token amount on the destination chain. It marks the message resolved, deletes cross-chain recovery metadata, transfers the tokens, and emits MessageRecoveredLocally.

Inputs

- messageId
 - **Control:** Full control.
 - **Constraints:** The ID must be in BASIC failed state, the stored payload must contain a nonzero local refund address equal to msg . sender , and at least one stored token amount must be positive.
 - **Impact:** Selects the failed message, authorized local recipient, and token balances transferred by the recovery.

Branches and code coverage

Intended branches

- ☒ The stored local refund address can recover the full positive token amount, mark the message RESOLVED, and emit MessageRecoveredLocally.
- ☐ Successful local recovery should directly assert deletion of the failed-message record and refund-family snapshot.
- ☐ Zero-amount entries should be skipped when at least one other stored entry is positive.
- ☒ Successful local recovery should make subsequent cross-chain recovery unavailable, and successful cross-chain recovery should make local recovery unavailable.

Negative behavior

- ☒ Revert when the message is not in BASIC failed state.
- ☒ Revert when the caller is not the stored local refund address.
- ☒ Revert when every stored token amount is zero.
- ☐ Revert when the failed message has no local refund address by directly exercising NoLocalRefundAddress.
- ☐ Revert atomically and preserve failed-message state when any stored token transfer fails.
- ☐ Exercise token callback reentrancy against the recovery function.

Function: recoverNative(address recipient)

This admin-only recovery function transfers the adapter's entire native-token balance to a selected recipient. That balance normally funds outbound CCIP return and refund fees, so recovery changes the adapter's ability to complete future cross-chain sends.

Inputs

- recipient
 - **Control:** Full control.
 - **Constraints:** The caller must hold DEFAULT_ADMIN_ROLE, the recipient must be nonzero, the adapter balance must be positive, and the recipient must accept the native-token transfer.
 - **Impact:** Receives all native tokens held by the adapter, including funds otherwise available for CCIP fees.

Branches and code coverage

Intended branches

- ☒ An authorized admin can transfer the full native balance to a valid recipient.

Negative behavior

- ☒ Revert when the caller lacks `DEFAULT_ADMIN_ROLE`.
- ☒ Revert when the recipient is zero.
- ☒ Revert when the adapter native balance is zero.
- ☒ Revert when the recipient rejects the native transfer.
- ☐ After a rejected transfer, directly assert that the adapter retains its full native balance.

Function: `refundFailedMessage(bytes32 messageId)`

This permissionless payable recovery function bridges every nonzero token amount from a failed inbound message back to its normalized original sender. It resolves and deletes the stored failure state before sending, pays each CCIP fee from the transaction's native value, refunds excess native value to the caller, and rolls the entire transaction back if any leg fails.

Inputs

- `messageId`
 - **Control:** Full control.
 - **Constraints:** The ID must be in BASIC failed state, its stored message must contain at least one positive token amount, its stored sender must be exactly 32 bytes, and its refund chain family must resolve to EVM or SVM.
 - **Impact:** Selects the stored source chain, original recipient, token balances, and encoding snapshot used for the refund.
- `msg.value`
 - **Control:** Full control.
 - **Constraints:** The supplied native value must cover the sum of fees actually quoted and paid by every nonzero refund leg. Any excess is returned to the caller, who must be able to receive it.
 - **Impact:** Funds outbound CCIP refund fees without changing the fixed refund recipient or token amounts.

Branches and code coverage

Intended branches

- ☒ A caller can fund a refund that sends the stored token amount to the original source sender and marks the message `RESOLVED`.
- ☒ Excess native value should be returned to the caller after the actual paid fee is calculated.
- ☒ A zero-amount entry should be skipped while a positive entry creates one outbound refund leg.

- ☐ Multiple positive entries should complete all outbound refund legs successfully in one transaction.
- ☒ An SVM failure snapshot should preserve SVM encoding even if the live chain classification is later changed to EVM.
- ☒ An unconfigured source selector should infer EVM or SVM encoding from a canonical 32-byte sender.

Negative behavior

- ☒ Revert when the message is not in BASIC failed state.
- ☒ Revert when every stored destination-token amount is zero.
- ☒ Revert when the stored sender is not exactly 32 bytes.
- ☒ Revert when sequential router fee increases make `msg.value` lower than the sum actually paid.
- ☒ Revert when the caller rejects the excess native-value refund.
- ☒ Revert after either local or cross-chain recovery has already resolved the message.
- ☐ Revert atomically when the refund chain family cannot be resolved, the router rejects the destination lane, or a token approval or send fails.
- ☐ Exercise a malicious token or caller callback against the nonreentrant recovery path.

Function: `sendToken(uint64 destinationChainSelector, bytes32 beneficiary, address token, uint256 amount, ChainType encodingOverride)`

This internal transport function builds and submits one outbound CCIP token message. It resolves the EVM or SVM wire family, quotes the router fee, checks the adapter's native balance, approves the router for the token amount, sends the message, and emits the resulting message ID and fee.

Inputs

- `destinationChainSelector`
 - **Control:** Indirect control. A normal return uses the authenticated inbound source-chain selector, while a refund uses the selector stored from the failed inbound message.
 - **Constraints:** Return paths derive the selector from the authenticated inbound source and require its live chain type to be enabled. Refund paths derive it from stored failed-message data and may use a snapshotted encoding override. The router must support the destination lane.
 - **Impact:** Selects the CCIP destination and fee quote for the outbound token transfer.
- `beneficiary`
 - **Control:** Partial control. The remote sender selects the beneficiary for a normal return, while a refund fixes it to the authenticated sender stored from

- the failed inbound message.
 - **Constraints:** Return paths derive the value from the source payload, while refund paths use the normalized stored sender. EVM sends require a canonical right-aligned address; SVM sends use the full 32-byte value as the token receiver.
 - **Impact:** Identifies the account that receives the bridged token on the destination chain.
- token
 - **Control:** Indirect control. A normal return derives the token from the executed vault action, while a refund uses a token delivered by CCIP and stored in the failed-message record.
 - **Constraints:** The value is the vault-action output token or a token stored from the failed inbound message, and it must be supported by the router and destination lane.
 - **Impact:** Selects the ERC-20 approved to and bridged by the CCIP router.
- amount
 - **Control:** Indirect control. A normal return derives the amount from the vault action's output, while a refund uses the delivered amount stored in the failed-message record.
 - **Constraints:** The value must be nonzero and must not exceed the adapter's available token balance.
 - **Impact:** Sets the token quantity bridged to the beneficiary.
- encodingOverride
 - **Control:** No direct control. Normal returns hardcode NONE; refunds derive the value from the failure-time snapshot, live chain configuration, or the stored sender encoding.
 - **Constraints:** NONE uses the live chains[destinationChainSelector] value. Refunds may pass an EVM or SVM snapshot, and the effective family must not be NONE.
 - **Impact:** Selects EVM or SVM receiver and extra-arguments encoding independently of later live chain reconfiguration.

Branches and code coverage

Intended branches

- ☒ An EVM return should encode a canonical EVM receiver and legacy V2 extra arguments by default.
- ☒ A configured EVM V3 return should encode the stored per-token requested finality.
- ☒ An SVM return should use the zero receiver placeholder and encode the 32-byte beneficiary as the SVM token receiver.
- ☒ Refunds should preserve a snapshotted SVM family after live chain reclassification and

should support inferred EVM and SVM families for previously unconfigured sources.

- ☒ A zero-valued refund entry should be skipped by the caller while a positive entry produces one outbound send.
- ☐ Multiple positive refund entries should each complete one outbound send in a successful transaction.

Negative behavior

- ☐ Revert when amount is zero.
- ☐ Revert when the effective chain family is NONE.
- ☐ Revert when an EVM beneficiary has nonzero upper 96 bits.
- ☐ Revert when the adapter lacks the native balance required by the current router quote.
- ☐ Propagate a token approval or router-send failure without retaining partial state or a reusable unintended allowance.

Function: `setCCVsConfig(uint64 sourceChainSelector, address[] requiredCCVs, address[] optionalCCVs, uint8 optionalThreshold_)`

This admin-only function stores the CCIP V2 verifier policy advertised for inbound messages from one source chain. It enforces a usable optional threshold and uniqueness across the required and optional verifier lists.

Inputs

- `sourceChainSelector`
 - **Control:** Full control.
 - **Constraints:** The caller must hold `DEFAULT_ADMIN_ROLE`. The selector does not have to be enabled in chains when the policy is staged.
 - **Impact:** Selects the inbound source chain whose CCV verification policy is replaced.
- `requiredCCVs`
 - **Control:** Full control.
 - **Constraints:** Entries must be unique within this list and must not also appear in `optionalCCVs`. The zero address is permitted as a required entry by the implementation.
 - **Impact:** Defines the CCVs that must always attest to inbound messages from the selected source chain.
- `optionalCCVs`
 - **Control:** Full control.
 - **Constraints:** Entries must be nonzero, unique within the list, and absent from `requiredCCVs`.

- **Impact:** Defines the additional CCVs from which the configured threshold may be satisfied.
- optionalThreshold_
 - **Control:** Full control.
 - **Constraints:** The threshold must not exceed optionalCCVs.length and must be positive when the optional list is nonempty.
 - **Impact:** Sets how many optional CCVs must attest for the advertised inbound policy.

Branches and code coverage

Intended branches

- ☒ An admin can store required and optional CCV lists, a valid threshold, and emit CCVsConfigSet.
- ☒ Policies for different source-chain selectors should remain isolated.
- ☒ An empty optional list may use a zero threshold.
- ☒ The required list may contain the zero-address placeholder accepted by the implementation.

Negative behavior

- ☒ Revert when the caller lacks DEFAULT_ADMIN_ROLE.
- ☒ Revert when the optional threshold exceeds the optional-list length.
- ☒ Revert when the optional list is nonempty but the threshold is zero.
- ☒ Revert when an optional CCV is the zero address.
- ☒ Revert when an address is duplicated within either list or appears in both lists.
- ☐ Exercise operationally large verifier arrays, and document any maximum size required to keep configuration and CCIP verification practical.

Function: setChainType(uint64 chainSelector, ChainType chainType)

This admin-only function classifies a CCIP chain selector as EVM, SVM, or disabled. The classification gates inbound processing and determines the receiver and extra-arguments encoding used by outbound return messages.

Inputs

- chainSelector
 - **Control:** Full control.
 - **Constraints:** The caller must hold DEFAULT_ADMIN_ROLE. The contract does not independently verify that the selector is supported by the configured

CCIP router.

- **Impact:** Selects the inbound and outbound CCIP lane whose family classification is updated.
- chainType
 - **Control:** Full control.
 - **Constraints:** The ABI value must be a valid ChainType enum member: NONE, EVM, or SVM.
 - **Impact:** Enables the selector and chooses its wire format or disables inbound processing when set to NONE.

Branches and code coverage

Intended branches

- ☒ An admin can configure an EVM selector and emit ChainTypeSet.
- ☒ EVM and SVM classifications should produce their corresponding outbound receiver and extra-arguments encodings.
- ☒ Reclassifying a source from SVM to EVM after a message fails should not change that message's snapshotted SVM refund encoding.
- ☐ Setting a previously enabled selector to NONE should disable future inbound processing without corrupting the encoding snapshot of an already failed message.

Negative behavior

- ☒ Revert when the caller lacks DEFAULT_ADMIN_ROLE.
- ☐ Exercise a validly encoded but router-unsupported selector, and verify that later transport failures remain recoverable.

Function: setEvmReturnLaneFormat(uint64 destinationChainSelector, Evm-ReturnExtraArgsFormat format)

This admin-only function selects whether outbound EVM return and refund messages for a destination lane use legacy CCIP V2 extra arguments or generic CCIP V3 basic extra arguments.

Inputs

- destinationChainSelector
 - **Control:** Full control.
 - **Constraints:** The caller must hold DEFAULT_ADMIN_ROLE. The function does not require the selector to be enabled in chains; the selected format is consumed only if an outbound message later uses that selector.
 - **Impact:** Selects the return or refund lane whose wire encoding is changed.

- `format`
 - **Control:** Full control.
 - **Constraints:** The value must be `LEGACY_EXTRA_ARGS_V2` or `GENERIC_EXTRA_ARGS_V3_BASIC`; `UNSET` is rejected.
 - **Impact:** Determines the extra-arguments format passed to the destination lane's CCIP on-ramp for future token sends.

Branches and code coverage

Intended branches

- ☒ Configuring the generic V3 format should cause a return message to use V3 basic extra arguments.
- ☒ An unset lane format should resolve to legacy V2 extra arguments for a return message.
- ☒ Explicitly configuring the legacy format should be accepted and cause a nonzero V3 requested-finality setting to revert.
- ☐ Exercise an explicitly configured legacy lane through an outbound send and assert the exact legacy V2 return-message encoding.
- ☐ A successful update should assert the stored format and the exact `EvmReturnLaneFormatSet` event payload.

Negative behavior

- ☒ Revert when format is `UNSET`.
- ☒ Revert when the caller lacks `DEFAULT_ADMIN_ROLE`.
- ☐ Exercise a selector that is not configured in chains, and verify that configuration staging does not itself enable the lane.

Function: `setEvmReturnRequestedFinality(uint64 destinationChainSelector, address token, bytes4 requestedFinalityForV3)`

This admin-only function configures the requested CCIP V3 finality for a token returned or refunded over an EVM lane. Legacy lanes may only clear the stored value, while V3 lanes validate and store one requested-finality mode.

Inputs

- `destinationChainSelector`
 - **Control:** Full control.
 - **Constraints:** The caller must hold `DEFAULT_ADMIN_ROLE`. The lane must resolve to the V3 basic format to store a nonzero requested finality.

- **Impact:** Selects the outbound EVM lane whose per-token finality request is updated.
- token
 - **Control:** Full control.
 - **Constraints:** The token address must be nonzero.
 - **Impact:** Selects which bridged token uses the configured finality on the chosen lane.
- requestedFinalityForV3
 - **Control:** Full control.
 - **Constraints:** On a legacy lane, the value must be zero and clears storage. On a V3 lane, FinalityCodec requires finality, one flag, or one block-depth mode rather than a combination of modes.
 - **Impact:** Controls the finality requested from CCIP for future return and refund sends of the selected token.

Branches and code coverage

Intended branches

- ☒ A valid finality value stored for a V3 lane should appear in the encoded return-message extra arguments.
- ☐ Supplying zero on a legacy lane should delete a previously stored requested-finality value and emit the cleared value.
- ☐ Different tokens on the same destination selector should retain independent finality settings.

Negative behavior

- ☒ Revert when token is the zero address.
- ☒ Revert when a nonzero finality is supplied for a legacy lane.
- ☐ Revert when the caller lacks DEFAULT_ADMIN_ROLE.
- ☐ Revert when the requested finality combines more than one mode.

Function: setInboundFinality(uint64 sourceChainSelector, bytes4 allowedFinalityConfig)

This admin-only function stores the CCIP V2 allowed-finality policy advertised for inbound messages from a selected source chain.

Inputs

- sourceChainSelector

- **Control:** Full control.
- **Constraints:** The caller must hold `DEFAULT_ADMIN_ROLE`. The selector does not have to be enabled in chains when the policy is staged.
- **Impact:** Selects the inbound source chain whose advertised finality policy is updated.
- `allowedFinalityConfig`
 - **Control:** Full control.
 - **Constraints:** The adapter stores the four bytes without local validation; operators must encode a policy understood by the CCIP V2 verifier system.
 - **Impact:** Defines the finality modes that CCIP may accept for inbound messages from the selected source chain.

Branches and code coverage

Intended branches

- ☒ An admin can update one source chain's finality, emit `InboundFinalitySet`, and retrieve the stored value through the CCIP V2 configuration interface.
- ☒ Updating one selector should not alter another selector's finality.
- ☐ Composite allowed-finality encodings and zero-value resets should be stored and surfaced unchanged.

Negative behavior

- ☒ Revert when the caller lacks `DEFAULT_ADMIN_ROLE`.
- ☐ Exercise a malformed or unsupported finality bit pattern, and document whether downstream CCIP components reject it.

Function: `setProcessingEnabled(bool depositsEnabled_, bool redeemsEnabled_)`

This admin-only circuit-breaker function independently enables or disables deposit-related and redemption-related vault actions. Deposits and cooldown cancellation use the deposit flag, while cooldown creation and unstaking use the redeem flag.

Inputs

- `depositsEnabled_`
 - **Control:** Full control.
 - **Constraints:** The caller must hold `DEFAULT_ADMIN_ROLE`.
 - **Impact:** Enables or disables `DEPOSIT` and `CANCEL_COOLDOWN` actions and their corresponding previews.

- `redeemsEnabled_`
 - **Control:** Full control.
 - **Constraints:** The caller must hold `DEFAULT_ADMIN_ROLE`.
 - **Impact:** Enables or disables `COOLDOWN` and `UNSTAKE` actions and share-redemption previews.

Branches and code coverage

Intended branches

- ☒ Enabling both flags should allow deposit, cooldown, unstake, and cancellation flows.
- ☒ Disabling deposits should make the deposit preview revert, and disabling redemptions should make the redeem preview revert.
- ☐ Toggling either flag independently should leave the opposite preview and action paths available.
- ☐ A successful update should assert both stored values and the exact `ProcessingEnabledSet` event payload.

Negative behavior

- ☒ Revert when the caller lacks `DEFAULT_ADMIN_ROLE`.
- ☒ A disabled deposit path should make an inbound deposit fail and enter failed-message recovery.
- ☐ A disabled redeem flag should make both inbound `COOLDOWN` and `UNSTAKE` fail, and a disabled deposit flag should make `CANCEL_COOLDOWN` fail, without consuming cooldown state.

5.3. GenericMorphoAdapter.sol

Function: `allocate(bytes data, uint256 assets, bytes4 selector, address sender)`

This parent-vault-only function invests allocated underlying assets into the configured ERC-4626 vault. It decodes the allocator's minimum-share limit, deposits nonzero assets, checks the received shares, revalues the adapter, and returns the adapter allocation ID and accounting change to the parent Vault V2.

Inputs

- `data`
 - **Control:** Full control.
 - **Constraints:** The bytes must ABI-decode as one `uint256` value representing `minShares`; ABI-decodable trailing bytes are ignored.

- **Impact:** Sets the minimum number of ERC-4626 shares the authorized allocation is willing to accept.
- assets
 - **Control:** Full control.
 - **Constraints:** The direct caller must be `parentVault`. The parent vault must have transferred this amount of the shared underlying asset to the adapter and must satisfy its allocator, adapter, and cap checks.
 - **Impact:** Determines how many underlying tokens are deposited into the ERC-4626 vault and therefore changes the parent vault's exposure and reported allocation.
- selector
 - **Control:** Indirect control.
 - **Constraints:** The transaction caller can choose a parent-vault entry point that reaches `allocateInternal`, but the parent Vault V2 supplies the value as its current `msg.sig`. The value is therefore fixed to the selector of `allocate(address,bytes,uint256)`, `deposit(uint256,address)`, or `mint(uint256,address)` and cannot be chosen independently. The adapter does not read it.
 - **Impact:** Has no effect in this adapter implementation.
- sender
 - **Control:** Indirect control.
 - **Constraints:** The parent Vault V2 supplies the original transaction caller as `msg.sender`. An allocator controls this value when calling `allocate`, while a depositor or minter supplies it when allocation is triggered through the configured liquidity adapter. It cannot be set independently of the calling account, and the adapter does not read it.
 - **Impact:** Has no effect in this adapter implementation.

Branches and code coverage

Intended branches

- ☒ A parent-vault allocation with nonzero assets should deposit the assets, retain the resulting ERC-4626 shares in the adapter, and update the parent allocation by the reported value change.
- ☒ Allocations into the configured test vault and two production ERC-4626 integrations should report the resulting underlying value.
- ☐ A zero-asset allocation should skip the ERC-4626 deposit and return only any revaluation delta.
- ☐ The ignored selector and sender values should be varied to confirm that they cannot affect accounting.

Negative behavior

- ☒ Revert when the caller is not the configured parent vault.
- ☒ Revert when the ERC-4626 vault returns fewer shares than `minShares`.
- ☐ Revert when data is too short or otherwise malformed, and explicitly document that ABI-decodable trailing bytes are ignored.
- ☐ Propagate an ERC-4626 deposit failure without leaving transferred underlying or parent allocation state partially updated.

Function: `deallocate(bytes data, uint256 assets, bytes4 selector, address sender)`

This parent-vault-only function withdraws a requested amount of underlying from the configured ERC-4626 vault into the adapter. It decodes the allocator's maximum-share limit, checks the shares burned by the withdrawal, revalues the remaining position, and returns the allocation ID and accounting change before the parent vault pulls the recovered underlying.

Inputs

- `data`
 - **Control:** Full control.
 - **Constraints:** The bytes must ABI-decode as one `uint256` value representing `maxShares`; ABI-decodable trailing bytes are ignored.
 - **Impact:** Caps how many ERC-4626 shares may be burned to recover the requested underlying assets.
- `assets`
 - **Control:** Full control.
 - **Constraints:** The direct caller must be `parentVault`, and the configured ERC-4626 position must be able to withdraw the requested amount to the adapter.
 - **Impact:** Determines the amount of underlying removed from the ERC-4626 vault and subsequently pulled back into the parent vault.
- `selector`
 - **Control:** Indirect control.
 - **Constraints:** The transaction caller can choose a parent-vault entry point that reaches `deallocateInternal`, but the parent Vault V2 supplies the value as its current `msg.sig`. The value is therefore fixed to the selector of `deallocate(address,bytes,uint256)`, `withdraw(uint256,address,address)`, `redeem(uint256,address,address)`, or `forceDeallocate(address,bytes,uint256,address)` and cannot be chosen independently. The adapter does not read it.

- **Impact:** Has no effect in this adapter implementation.
- sender
 - **Control:** Indirect control.
 - **Constraints:** The parent Vault V2 supplies the original transaction caller as `msg.sender`. The value may identify an allocator or sentinel calling `deallocate`, a withdrawer or redeemer using the liquidity adapter, or a caller of `forceDeallocate`; it cannot be set independently of the calling account. The adapter does not read it.
 - **Impact:** Has no effect in this adapter implementation.

Branches and code coverage

Intended branches

- ☒ A parent-vault deallocation with nonzero assets should withdraw the requested underlying, reduce the adapter share balance, and update the parent allocation by the reported value change.
- ☒ Deallocation from the configured test vault and two production ERC-4626 integrations should recover assets within the accepted share limit.
- ☐ A zero-asset deallocation should skip the ERC-4626 withdrawal and return only any revaluation delta.
- ☐ The ignored `selector` and `sender` values should be varied to confirm that they cannot affect accounting.

Negative behavior

- ☒ Revert when the caller is not the configured parent vault.
- ☒ Revert when the ERC-4626 vault burns more shares than `maxShares`.
- ☐ Revert when data is too short or otherwise malformed, and explicitly document that ABI-decodable trailing bytes are ignored.
- ☐ Propagate an ERC-4626 withdrawal failure without allowing the parent vault to pull assets or commit an allocation change.

Function: `realAssets()`

This accounting function converts the adapter's full ERC-4626 share balance into underlying-asset units and reports that value to the parent Vault V2. The returned value drives allocation deltas and therefore affects the parent vault's view of its total managed assets.

Inputs

This function takes no explicit inputs.

Branches and code coverage

Intended branches

- ☒ A positive share balance in the configured test vault should be converted into the expected underlying-asset value after allocation and deallocation.
- ☒ Share balances in the sUSDS and static aToken ERC-4626 integrations should be reported within the configured rounding tolerance.
- ☐ A zero share balance should return zero.
- ☐ Yield, loss, donation, and ERC-4626 rounding changes should be reflected in the reported parent-vault allocation.

Negative behavior

- ☐ A fee-charging or otherwise nonstandard ERC-4626 vault should be tested to determine whether `convertToAssets` represents realizable redemption proceeds.
- ☐ Propagate a revert from `balanceOf` or `convertToAssets` without committing a parent-vault accounting update.

Function: `rescueToken(address token, address to)`

This owner-only recovery function transfers the adapter's entire balance of a selected ERC-20 token to a recipient. It prohibits rescuing the configured ERC-4626 share token so the owner cannot remove the position that backs the parent vault's reported allocation.

Inputs

- `token`
 - **Control:** Full control.
 - **Constraints:** The caller must be the owner, and `token` must not equal the configured ERC-4626 vault share token.
 - **Impact:** Selects the ERC-20 balance removed in full from the adapter. This may include accidentally sent tokens or uninvested underlying assets.
- `to`
 - **Control:** Full control.
 - **Constraints:** The contract does not validate the recipient; the selected token's transfer implementation determines whether a zero or otherwise invalid recipient is accepted.
 - **Impact:** Receives the adapter's complete balance of the selected token.

Branches and code coverage

Intended branches

- ☒ The owner can recover the full balance of an unrelated ERC-20 token.

Negative behavior

- ☒ Revert when the selected token is the configured ERC-4626 share token.
- ☐ Revert when the caller is not the owner.
- ☐ Exercise recovery of the underlying asset, and document whether removing uninvested parent-vault assets is intended.
- ☐ Exercise a zero recipient and a token that returns false or reverts during transfer.

5.4. GenericMorphoAdapterFactory.sol

Function: `createGenericMorphoAdapter(address parentVault, address vault, address owner)`

This permissionless factory function deploys a GenericMorphoAdapter with CREATE2 and a fixed salt, records the resulting adapter for the supplied parent-vault and ERC-4626-vault pair, marks the address as factory-created, emits a creation event, and returns the new address.

Inputs

- `parentVault`
 - **Control:** Full control.
 - **Constraints:** The address must support the Vault V2 calls used by the adapter constructor, and its underlying asset must match the ERC-4626 vault's asset.
 - **Impact:** Becomes the only contract allowed to allocate and deallocate through the deployed adapter and forms the first key of the factory lookup.
- `vault`
 - **Control:** Full control.
 - **Constraints:** The address must support the ERC-4626 calls used by the adapter, and its asset must equal the parent vault's asset.
 - **Impact:** Becomes the yield vault that receives underlying assets and issues shares, and forms the second key of the factory lookup.
- `owner`
 - **Control:** Full control.
 - **Constraints:** The address must be nonzero because the adapter inherits Ownable.
 - **Impact:** Receives authority over adapter ownership transfer and token recovery. It also changes the CREATE2 init-code hash and therefore the deployed address.

Branches and code coverage

Intended branches

- ☒ Valid inputs should deploy and return an adapter that exposes the expected parent vault, ERC-4626 vault, asset, and allocation ID.
- ☐ The deployed adapter should assign the supplied owner as its owner.
- ☐ The factory should record the returned address in both lookup mappings and emit `CreateGenericMorphoAdapter` with the deployed address.
- ☐ Deploying the same parent-vault and ERC-4626-vault pair with a different owner should explicitly test and document whether replacing the pair lookup is intended.

Negative behavior

- ☐ Revert when the exact constructor arguments have already been deployed and the fixed-salt `CREATE2` address already contains code.
- ☐ Revert through the factory when the parent vault and ERC-4626 vault report different assets.
- ☐ Revert for zero or noncontract parent-vault and vault addresses and for a zero owner.
- ☐ Reject or explicitly document `parentVault == vault`, which makes the parent vault account for a position composed of its own shares.

6. Assessment Results

During our assessment on the scoped Tenbin contracts, we discovered seven findings. No critical issues were found. Two findings were of medium impact, one was of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.